

Final Project Documentation

Pet Care E-Commerce Platform

Team Information

- Project Title: Pet Care E-Commerce Platform
 - Team Members:
 - Mostafa Magdy Ahmed (Frontend-UI/UX)
 - Menna Mohamed Hassan (Frontend)
 - Omnia Maged Ragab (Frontend)
 - Toka Mohamed Elalfy (Backend)
 - Mohamed Ebrahim Mohamed (Backend)
 - Rawaa Ahmed Atia (Backend)
 - Instructor: (Ebrahim El-tbakh)
 - Date: 2026
-

Table of Contents

1. Project Planning & Management
2. Requirements Gathering
3. System Analysis & Design
4. Implementation
5. Testing & Quality Assurance
6. Final Report

1. Project Planning & Management

1.1 Project Proposal

Overview

The Pet Care E-Commerce Platform is a modern web-based application designed to enhance the online shopping experience for pet owners. Unlike traditional e-commerce platforms, this system provides a personalized experience tailored to each user's pet by utilizing specific pet-related data such as type, age, weight, and breed.

The platform enables users to create and manage pet profiles, which are then used to dynamically filter and recommend suitable products. It also incorporates real-world features such as subscription-based purchasing and automated reorder reminders based on consumption patterns. The system focuses on usability, efficiency, and practical real-life applications using structured logic rather than artificial intelligence.

Objectives

The main objectives of this project are:

- Develop a complete and scalable e-commerce platform dedicated to pet products
 - Provide a personalized shopping experience based on pet characteristics
 - Implement subscription-based purchasing for recurring essential items
 - Introduce a smart reorder reminder system to improve user convenience
 - Apply full-stack development concepts using modern web technologies
 - Build a user-friendly and responsive interface
-

Scope

The system will include the following functionalities:

User Management

- User registration and login
- Authentication and authorization

Pet Profile Management

- Add, update, and delete pet profiles
- Store pet details (type, age, weight, breed)

Product Management

- Browse and search products
- Dynamic filtering based on pet characteristics

Order Management

- Add products to cart
- Place and track orders

Subscription System

- Subscribe to recurring products
- Control delivery frequency and quantity
- Pause or cancel subscriptions

Reorder Reminder System

- Notify users before products run out based on usage patterns

User Dashboard

- Manage pets, orders, subscriptions, and notifications
-

Target Users

The platform is designed for:

- Pet owners (dogs, cats, and other animals)
 - First-time pet adopters
 - Users who regularly purchase pet food and care products
-

Key Features

- Pet Profile System for personalization
 - Smart product filtering based on pet data
 - Subscription system for recurring purchases
 - Automated reorder reminders
 - Bundle builder for customized product packages
 - Centralized user dashboard
-

Technology Stack

Frontend

- React.js
- Responsive UI design
- State management

Backend

- Node.js
- Express.js
- RESTful APIs

Database

- MongoDB
-

Expected Outcomes

- A fully functional pet-focused e-commerce platform
 - Personalized shopping experience based on real user data
 - Implementation of real-world business logic
 - Demonstration of full-stack development skills
-

Project Value

This project demonstrates:

- Strong system design and architecture skills
 - Ability to build scalable and maintainable applications
 - Integration between frontend and backend systems
 - Practical implementation of e-commerce workflows
-

Future Enhancements

- Online payment integration
 - Veterinary appointment booking system
 - Pet health tracking tools
 - Mobile application version
-

1.2 Project Plan (Timeline & Gantt Chart)

Project Duration

Estimated project duration: 12 Weeks

Timeline

Phase	Task	Duration	Start	End
Planning	Project Proposal	1 week	Week 1	Week 1
Planning	Project Plan & Roles	1 week	Week 2	Week 2
Analysis	Requirements Gathering	2 weeks	Week 3	Week 4
Design	System Design & Diagrams	2 weeks	Week 5	Week 6
Implementation	Frontend Development	2 weeks	Week 7	Week 8
Implementation	Backend Development	2 weeks	Week 7	Week 8
Integration	System Integration	1 week	Week 9	Week 9
Testing	Testing & Bug Fixing	1 week	Week 10	Week 10
Deployment	Deployment & Final Setup	1 week	Week 11	Week 11
Finalization	Documentation & Presentation	1 week	Week 12	Week 12

Milestones

- Week 2: Project Planning Completed
- Week 4: Requirements Approved
- Week 6: System Design Completed
- Week 8: Core Features Implemented
- Week 10: Testing Completed
- Week 12: Final Submission Ready

Deliverables

- Project Documentation
- Source Code (Frontend & Backend)
- Database Design
- Working Application
- Final Presentation

1.3 Roles & Responsibilities

Team Structure

The project will be developed by a team of 4–6 members, with each member assigned specific roles and responsibilities to ensure efficient workflow and accountability.

Roles Distribution

Role	Team Member(s)	Responsibilities
Project Manager	Toka Mohamed Elalfy	Planning, task distribution, timeline tracking, communication with instructor, ensuring deadlines are met
Frontend Developer	Mostafa Magdy Ahmed, Toka Mohammed Elalfy, Menna Mohamed Hassan	Building UI using React.js, implementing responsive design, handling state management, integrating APIs
Backend Developer	Mohamed Ebrahim Mohamed, Rawaa Ahmed Atia	Developing REST APIs, handling business logic, authentication, and server-side operations
UI/UX Designer	Omnia Maged Ragab	Creating wireframes and mockups, improving user experience, ensuring usability and consistency
Database Engineer	Mohamed Ebrahim Mohamed	Designing MongoDB schema, managing data relationships, optimizing queries
QA Tester	Team Members (Shared Responsibility)	Writing test cases, performing testing, reporting bugs, ensuring system quality

Responsibilities Details

- Each team member is responsible for completing assigned tasks within the timeline.
 - Regular team meetings will be conducted to track progress.
 - Code collaboration will be managed using GitHub with proper version control practices.
 - Each member will contribute to documentation and final presentation.
-

Collaboration Tools

- GitHub for version control and collaboration
 - Trello / Notion for task management
 - WhatsApp / Slack for team communication
-

1.4 Risk Management

Risk Identification & Mitigation Plan

Risk	Description	Impact	Mitigation Strategy
Poor Time Management	Delays in completing tasks due to poor planning	High	Use clear timeline, weekly meetings, and task tracking tools (Trello/Notion)
Team Communication Issues	Miscommunication between team members	Medium	Use consistent communication channels and regular meetings
Technical Challenges	Difficulty in implementing features (e.g., subscriptions, filtering)	High	Research solutions early, divide tasks, and seek mentor support
Integration Issues	Difficulty integrating frontend with backend systems	High	Test APIs early and integrate gradually
Data Loss	Potential data loss or database failures	Medium	Use backups and version control (GitHub)
GitHub Conflicts	Code conflicts between team members	Medium	Follow branching strategy and code reviews

Performance Issues	System slowdown with increasing data	Medium	Optimize queries and use efficient coding practices
UI/UX Problems	Poor user interface or user experience design	Low	Test UI with users and improve based on feedback

Risk Management Strategy

- Continuous monitoring of project progress
 - Weekly evaluation of risks and issues
 - Early detection and quick resolution of problems
 - Clear communication between all team members
-

1.5 KPIs (Key Performance Indicators)

KPIs Definition

Key Performance Indicators (KPIs) are used to measure the success and performance of the system based on usability, performance, and user engagement.

KPIs Metrics

KPI	Description	Target
System Response Time	Time taken for the system to respond to user requests	Less than 2 seconds
System Uptime	Availability of the system without downtime	99% uptime
User Satisfaction	User feedback based on experience	≥ 4/5 rating
Task Completion Rate	Percentage of users completing actions (e.g., placing order)	≥ 90%
Bug Rate	Number of bugs detected during testing	Low
Page Load Time	Time required to load pages	Less than 3 seconds
API Response Time	API response speed	Less than 500 ms

KPI Evaluation Strategy

- Monitor system performance using testing tools
- Collect user feedback during testing phase
- Track system logs and response times
- Continuously improve based on collected data

2. Requirements Gathering

2.1 Stakeholder Analysis

Introduction

Stakeholders are individuals or groups who are directly or indirectly affected by the system. Identifying stakeholders helps in understanding system requirements and ensuring that the platform meets user needs effectively.

Stakeholders Identification

Stakeholder	Description	Needs / Expectations
Pet Owners (Users)	Main users of the system	Easy product search, personalized recommendations, fast checkout
Admin	Manages products, users, and orders	Control panel, manage inventory, monitor system activity
Development Team	Builds and maintains the system	Clear requirements, maintainable code, scalability
Instructor	Evaluates the project	Complete documentation, working system, clean implementation
Business Owner	Interested in profitability and growth	Increased sales, user retention, efficient system

Stakeholder Importance

- Pet owners are the primary stakeholders, as the system is designed mainly for them.
- Admin users ensure the system runs efficiently.
- The development team ensures technical implementation.
- The instructor evaluates the overall project quality.
- The business owner focuses on the system's success and scalability.

2.2 User Stories

Introduction

User stories describe how different users interact with the system. They are written from the user's perspective and help define system functionality in a simple and clear way.

User Stories List

Account Management

- As a user, I want to create an account so that I can access the platform.
 - As a user, I want to log in to my account so that I can manage my data.
 - As a user, I want to log out securely to protect my account.
-

Pet Profile Management

- As a user, I want to add my pet's details (type, age, weight, breed) so that I can receive personalized recommendations.
 - As a user, I want to update my pet's information so that the system reflects accurate data.
 - As a user, I want to delete a pet profile if I no longer need it.
-

Product Browsing & Filtering

- As a user, I want to browse products so that I can find items for my pet.
 - As a user, I want to filter products based on my pet's characteristics so that I see relevant items.
 - As a user, I want to search for products by name or category.
-

Shopping & Orders

- As a user, I want to add products to my cart so that I can purchase them later.
 - As a user, I want to place an order so that I can receive products.
 - As a user, I want to view my order history so that I can track previous purchases.
-

Subscription System

- As a user, I want to subscribe to essential products so that I receive them automatically.
 - As a user, I want to choose delivery frequency (weekly/monthly).
 - As a user, I want to pause or cancel my subscription anytime.
-

Reorder Reminder

- As a user, I want to receive notifications before products run out so that I can reorder in time.
-

Dashboard

- As a user, I want to manage my pets, orders, and subscriptions in one place.
-

Admin Features

- As an admin, I want to manage products so that I can update inventory.
- As an admin, I want to view users and orders to monitor system activity.

2.3 Functional Requirements

Introduction

Functional requirements define the specific features and functions that the system must provide. These requirements describe what the system should do to meet user needs.

User Management

- The system shall allow users to register using email and password.
 - The system shall allow users to log in and log out securely.
 - The system shall support authentication and authorization.
-

Pet Profile Management

- The system shall allow users to create pet profiles.
 - The system shall store pet details (type, age, weight, breed).
 - The system shall allow users to update pet information.
 - The system shall allow users to delete pet profiles.
-

Product Management

- The system shall display a list of available products.
 - The system shall allow users to search for products.
 - The system shall allow filtering products based on pet characteristics.
 - The system shall categorize products (food, toys, grooming, etc.).
-

Shopping Cart & Orders

- The system shall allow users to add products to the cart.
 - The system shall allow users to update or remove items from the cart.
 - The system shall allow users to place orders.
 - The system shall store order details and history.
-

Subscription System

- The system shall allow users to subscribe to products.
 - The system shall allow users to set delivery frequency.
 - The system shall allow users to modify or cancel subscriptions.
 - The system shall automatically generate recurring orders.
-

Notification System

- The system shall send reminders for product reordering.
 - The system shall notify users about subscription updates.
-

User Dashboard

- The system shall provide a dashboard for managing pets, orders, and subscriptions.
 - The system shall display user activity and history.
-

Admin Panel

- The system shall allow admins to manage products.
- The system shall allow admins to view and manage users.
- The system shall allow admins to monitor orders and system activity.

2.4 Non-Functional Requirements

Introduction

Non-functional requirements define the quality attributes of the system such as performance, security, usability, and reliability. These requirements describe how the system should perform rather than what it should do.

Performance Requirements

- The system shall respond to user requests within 2 seconds.
 - The system shall support multiple users simultaneously without performance degradation.
 - The system shall load pages within 3 seconds.
-

Security Requirements

- The system shall ensure secure user authentication.
 - The system shall encrypt sensitive data such as passwords.
 - The system shall protect against unauthorized access.
-

Usability Requirements

- The system shall provide a user-friendly interface.
 - The system shall be easy to navigate for all users.
 - The system shall be responsive across different devices (mobile, tablet, desktop).
-

Reliability Requirements

- The system shall be available 99% of the time.
 - The system shall handle errors gracefully without crashing.
 - The system shall ensure data consistency and integrity.
-

Scalability Requirements

- The system shall support future growth in users and data.
- The system shall be designed to handle increased load efficiently.

Maintainability Requirements

- The system shall be modular and easy to maintain.
 - The system shall follow clean coding standards.
 - The system shall allow easy updates and feature additions.
-

Compatibility Requirements

- The system shall be compatible with modern web browsers (Chrome, Edge, Firefox).
 - The system shall work across different operating systems.
-

3. System Analysis & Design

3.1 Problem Statement & Objectives

3.1.1 Use Case Diagram & Descriptions

Introduction

Use Case Diagrams represent the interactions between actors and the system. They illustrate the main functionalities provided by the platform and show how different users interact with the system.

The following diagrams represent the main system actors and their interactions within the Pet Care E-Commerce Platform.

3.1.1.1 User Use Case Diagram

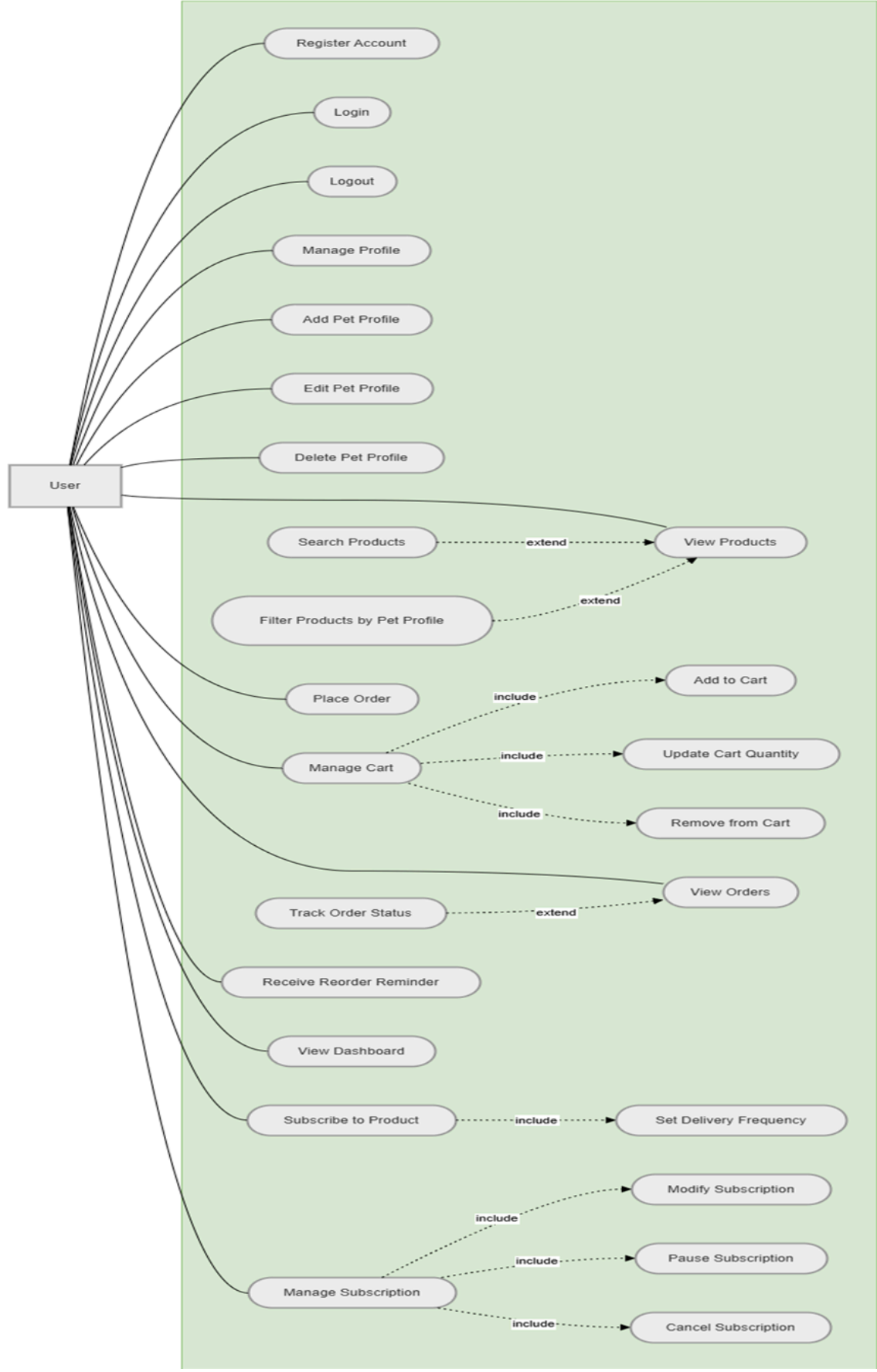
Description

This diagram illustrates the interactions between the user and the system. It covers all functionalities available for pet owners, including account management, pet profile management, product browsing, shopping cart operations, subscription handling, and dashboard access.

Main Functionalities

- User registration and authentication
- Pet profile management
- Product browsing and filtering
- Shopping cart management
- Order placement and tracking
- Subscription management
- Reorder reminder handling
- Dashboard access

Diagram:



3.1.1.2 Admin Use Case Diagram

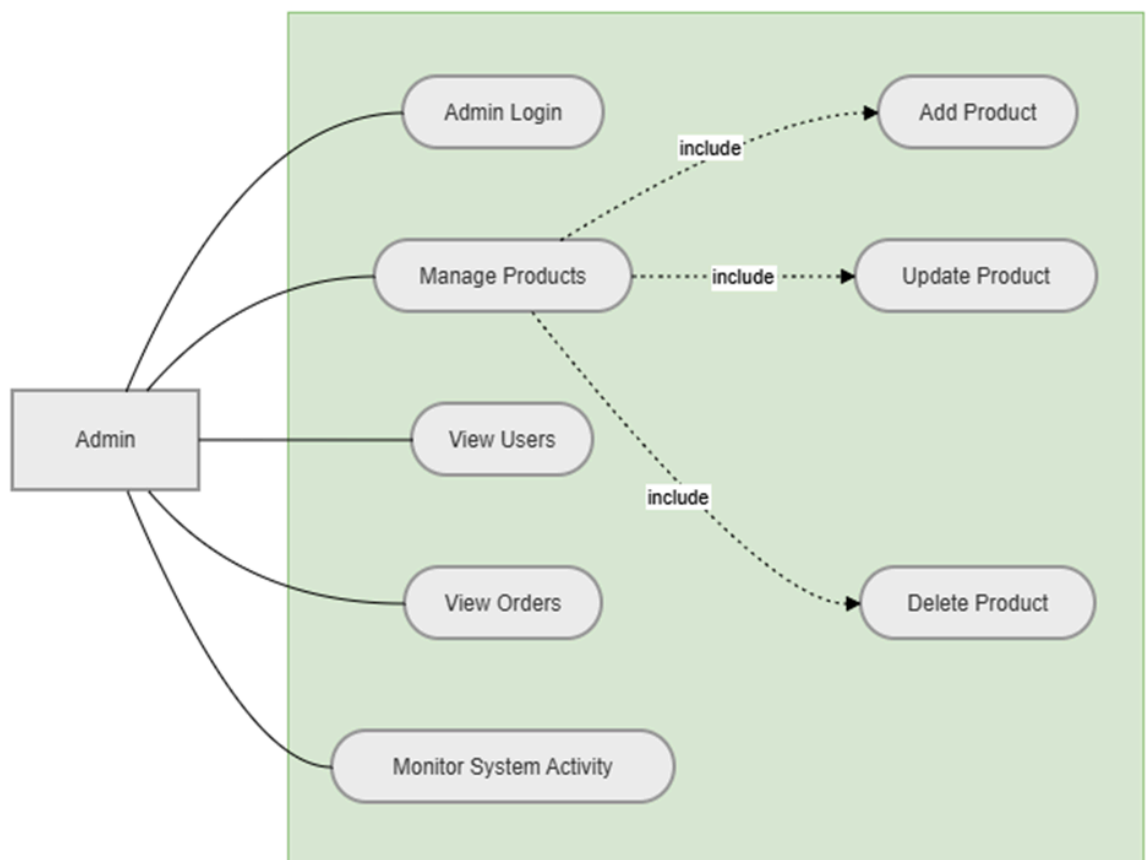
Description

This diagram represents the functionalities available for administrators. Admin users are responsible for managing products, monitoring orders, viewing users, and controlling overall system activities.

Main Functionalities

- Admin authentication
- Product management
- User monitoring
- Order monitoring
- System activity monitoring

Diagram :



3.1.1.3 Automated System Use Case Diagram

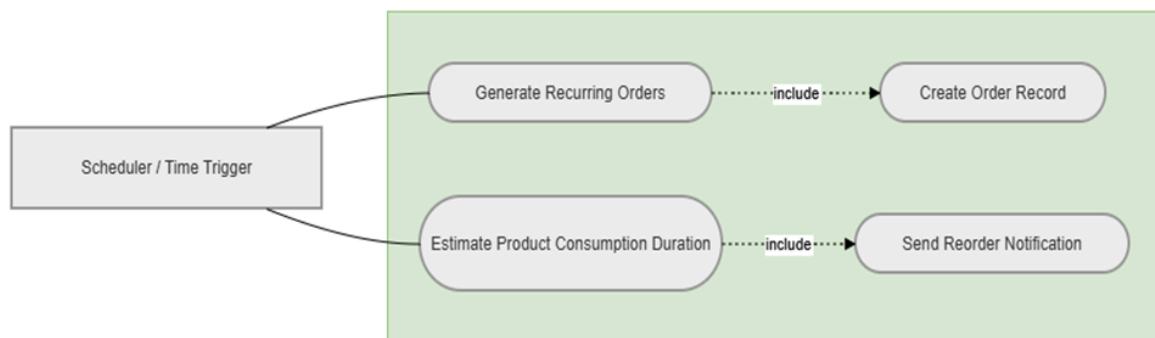
Description

This diagram illustrates automated background processes handled by the system scheduler. These automated operations support subscription management and reorder reminder functionality.

Main Functionalities

- Automatic recurring order generation
 - Creation of recurring order records
 - Estimation of product consumption duration
 - Sending reorder notifications
-

Diagram :



3.1.2 Functional & Non-Functional Requirements

Overview

The functional and non-functional requirements of the system were identified during the requirements gathering phase. These requirements define the system capabilities, operational behavior, and quality attributes.

Functional Requirements Summary

The system provides the following core functionalities:

- User registration and authentication
- Pet profile management
- Product browsing and filtering
- Shopping cart and order management
- Subscription management
- Reorder reminder notifications
- User dashboard management
- Admin product and user management

Non-Functional Requirements Summary

The system is designed to meet the following quality requirements:

- High performance and fast response time
- Secure authentication and data protection
- User-friendly and responsive interface
- Reliable and scalable architecture
- Maintainable and modular code structure
- Cross-browser and cross-device compatibility

Reference

Detailed functional and non-functional requirements are documented in Section 2: Requirements Gathering.

3.1.3 Software Architecture

Introduction

The software architecture defines the overall structure of the system, including its main components, interactions, and architectural style. The Pet Care E-Commerce Platform follows a layered MVC-inspired architecture combined with RESTful API communication between the frontend and backend systems.

This architecture ensures scalability, maintainability, modularity, and separation of concerns between system components.

Architecture Style

The system follows a client-server architecture using the MVC (Model-View-Controller) design pattern.

MVC Architecture Overview

- **Model** → Handles database operations and business data logic.
- **View** → Represents the frontend user interface displayed to users.
- **Controller** → Processes requests, applies business logic, and communicates between the frontend and database.

This architecture improves code organization and simplifies future maintenance and scalability.

System Components

The system consists of three major layers:

1. Frontend Layer

Description

The frontend layer is responsible for user interaction and interface presentation.

Technologies Used

- React.js
- HTML5
- CSS3
- JavaScript

Responsibilities

- Displaying user interfaces
- Handling user interactions
- Sending requests to backend APIs
- Managing application state
- Displaying personalized product recommendations

Main Frontend Modules

- Authentication Pages
 - Product Browsing Interface
 - Pet Profile Management Interface
 - Shopping Cart Interface
 - Subscription Management Interface
 - User Dashboard
-

2. Backend Layer

Description

The backend layer handles business logic, authentication, API processing, and communication with the database.

Technologies Used

- Node.js
- Express.js

Responsibilities

- Processing client requests
- Managing authentication and authorization
- Handling business logic
- Managing subscriptions and recurring orders
- Generating reorder reminders
- Communicating with the database

Main Backend Modules

Authentication Module : Handles user registration, login, and security validation.

Pet Management Module : Manages pet profiles and pet-related operations.

Product Management Module : Handles product retrieval, filtering, and management.

Order Management Module : Processes orders and manages order history.

Subscription Module : Handles recurring subscriptions and automated order generation.

Notification Module : Generates reorder reminders and subscription notifications.

Admin Management Module :Provides administrative control over products, users, and orders.

3. Database Layer

Description

The database layer stores and manages all system data.

Technology Used

- MongoDB

Responsibilities

- Storing user data
- Managing pet profiles
- Storing product information
- Managing orders and subscriptions
- Maintaining system data consistency

Main Collections

- Users
 - Pets
 - Products
 - Orders
 - Subscriptions
 - Notifications
-

System Interaction Flow

The interaction between system components follows the following process:

1. The user interacts with the frontend interface.
 2. The frontend sends HTTP requests to backend REST APIs.
 3. The backend validates requests and applies business logic.
 4. The backend communicates with the MongoDB database.
 5. The database returns requested data to the backend.
 6. The backend sends responses back to the frontend.
 7. The frontend displays processed data to the user.
-

RESTful API Communication

The system uses RESTful APIs for communication between the frontend and backend.

Example API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	User login
GET	/api/products	Retrieve products
POST	/api/orders	Create new order
GET	/api/pets	Retrieve pet profiles
POST	/api/subscriptions	Create subscription

Security Architecture

The system includes multiple security mechanisms to protect user data and system resources.

Security Features

- Password hashing and encryption
- Token-based authentication (JWT)
- Role-based access control for admin users
- Input validation and error handling

Scalability Considerations

The architecture is designed to support future scalability and expansion.

Scalability Features

- Modular backend structure
- Separation between frontend and backend
- Independent API services
- Flexible NoSQL database structure

This allows future integration of additional features such as online payment systems, veterinary booking services, and mobile applications.

Advantages of the Selected Architecture

- Clear separation of concerns
- Easier maintenance and debugging
- Improved scalability
- Better code organization
- Enhanced system security
- Easier future feature expansion

3.2.1 ER Diagram (Entity-Relationship Diagram)

Description

The Entity-Relationship Diagram (ERD) represents the database structure of the Pet Care E-Commerce Platform and illustrates the relationships between the main system entities.

The ERD is designed to support scalability, maintain data consistency, and efficiently manage relationships between users, pets, products, subscriptions, orders, and notifications.

The database follows a structured relational design where each entity represents a specific system component and relationships are established using primary and foreign keys.

Main Entities Overview

User Entity

Stores user account information including authentication and account details.

Pet Entity

Stores pet profile information associated with users to support personalized product recommendations.

Product Entity

Stores product details including category, animal type, pricing, and stock information.

Cart & Cart Item Entities

Manage shopping cart functionality and product quantities selected by users.

Order & Order Item Entities

Store order information and purchased products associated with each order.

Subscription Entity

Handles recurring product subscriptions and automated order scheduling.

Reminder Entity

Stores reorder reminder information and estimated product consumption tracking.

Bundle & Bundle Item Entities

Allow users to create custom product bundles with grouped products and pricing calculations.

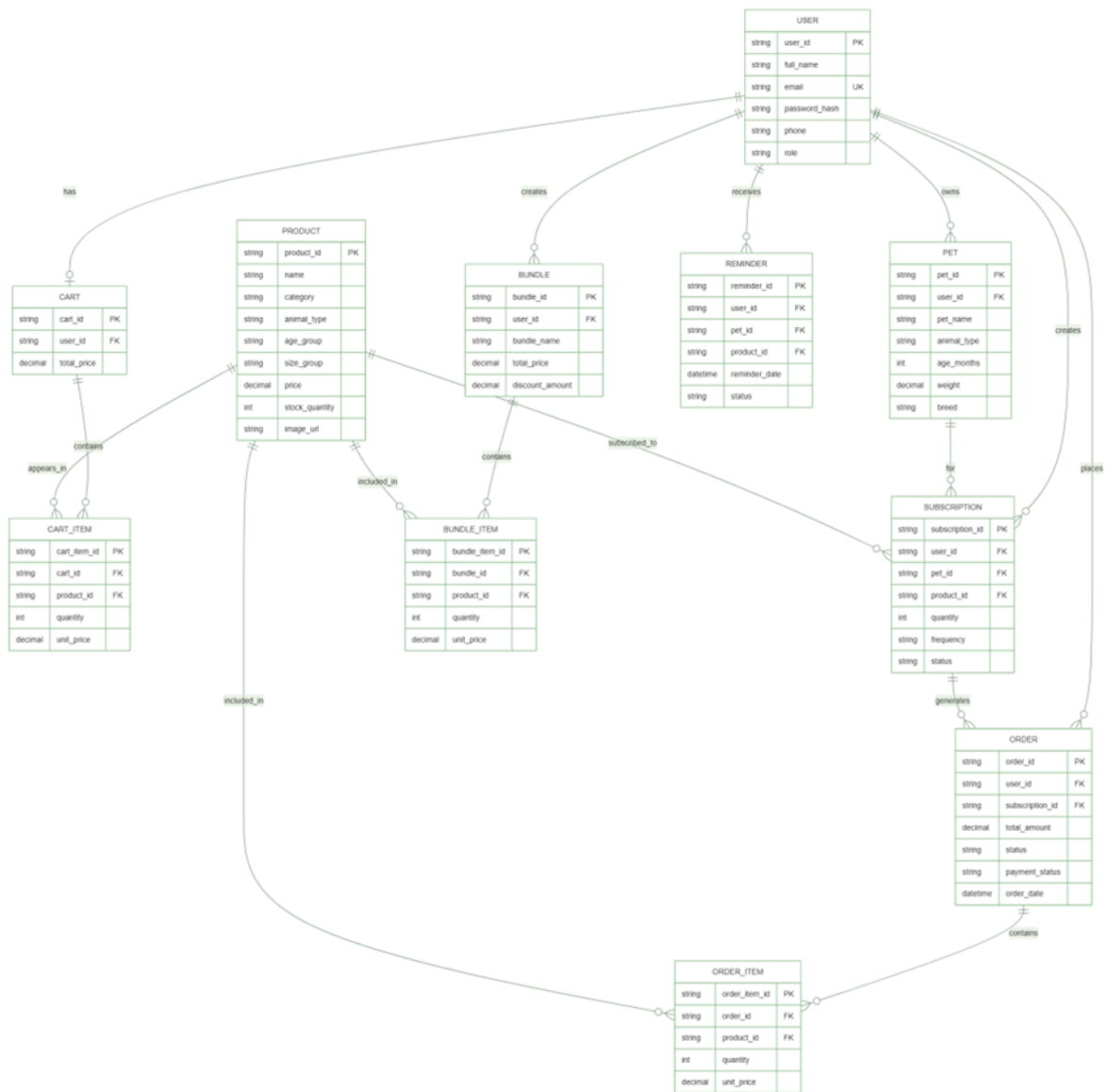
Relationship Overview

- One user can own multiple pets.
 - One user can create multiple orders and subscriptions.
 - Products can appear in carts, orders, bundles, and subscriptions.
 - Orders contain multiple order items.
 - Carts contain multiple cart items.
 - Bundles contain multiple bundle items.
 - Subscriptions are linked to users, pets, and products.
-

Design Considerations

- The database design minimizes data redundancy through normalized relationships.
- Junction tables are used to support many-to-many relationships.
- Foreign keys maintain data integrity between related entities.
- The schema is designed for scalability and future feature expansion.

Figure 3.4: Entity-Relationship Diagram



3.2.2 Logical & Physical Schema

Introduction

The logical and physical schema define the database structure of the Pet Care E-Commerce Platform. This includes tables (collections), attributes, primary keys, foreign keys, relationships, and normalization strategies used to ensure data consistency, scalability, and efficient system performance.

The database schema is designed based on the Entity-Relationship Diagram (ERD) and implemented using MongoDB collections.

Logical Schema

The logical schema defines the structure of the database entities and their relationships.

1. USER Collection

Description

Stores user account information and authentication data.

Attribute	Data Type	Key Type	Description
user_id	String	Primary Key	Unique identifier for each user
full_name	String	-	Full name of the user
email	String	Unique Key	User email address
password_hash	String	-	Encrypted password
phone	String	-	User phone number
role	String	-	User role (User/Admin)

2. PET Collection

Description

Stores pet profile information linked to users.

Attribute	Data Type	Key Type	Description
pet_id	String	Primary Key	Unique pet identifier
user_id	String	Foreign Key	References USER collection
pet_name	String	-	Pet name
animal_type	String	-	Type of animal
age_months	Integer	-	Pet age in months
weight	Decimal	-	Pet weight
breed	String	-	Pet breed

3. PRODUCT Collection

Description

Stores product information available in the platform.

Attribute	Data Type	Key Type	Description
product_id	String	Primary Key	Unique product identifier
name	String	-	Product name
category	String	-	Product category
animal_type	String	-	Target animal type
age_group	String	-	Target age group
size_group	String	-	Target size group
price	Decimal	-	Product price
stock_quantity	Integer	-	Available stock quantity

image_url	String	-	Product image URL
-----------	--------	---	-------------------

4. CART Collection

Description

Stores shopping cart information for users.

Attribute	Data Type	Key Type	Description
cart_id	String	Primary Key	Unique cart identifier
user_id	String	Foreign Key	References USER collection
total_price	Decimal	-	Total cart price

5. CART_ITEM Collection

Description

Stores products added to shopping carts.

Attribute	Data Type	Key Type	Description
cart_item_id	String	Primary Key	Unique cart item identifier
cart_id	String	Foreign Key	References CART collection
product_id	String	Foreign Key	References PRODUCT collection
quantity	Integer	-	Product quantity
unit_price	Decimal	-	Product price per unit

6. BUNDLE Collection

Description

Stores custom bundles created by users.

Attribute	Data Type	Key Type	Description
bundle_id	String	Primary Key	Unique bundle identifier
user_id	String	Foreign Key	References USER collection
bundle_name	String	-	Bundle name
total_price	Decimal	-	Bundle total price
discount_amount	Decimal	-	Applied discount amount

7. BUNDLE_ITEM Collection

Description

Stores products included in custom bundles.

Attribute	Data Type	Key Type	Description
bundle_item_id	String	Primary Key	Unique bundle item identifier
bundle_id	String	Foreign Key	References BUNDLE collection
product_id	String	Foreign Key	References PRODUCT collection
quantity	Integer	-	Product quantity
unit_price	Decimal	-	Product unit price

8. SUBSCRIPTION Collection

Description

Stores recurring subscription information for products.

Attribute	Data Type	Key Type	Description
subscription_id	String	Primary Key	Unique subscription identifier
user_id	String	Foreign Key	References USER collection
pet_id	String	Foreign Key	References PET collection

product_id	String	Foreign Key	References PRODUCT collection
quantity	Integer	-	Product quantity
frequency	String	-	Delivery frequency
status	String	-	Subscription status

9. REMINDER Collection

Description

Stores reorder reminder notifications.

Attribute	Data Type	Key Type	Description
reminder_id	String	Primary Key	Unique reminder identifier
user_id	String	Foreign Key	References USER collection
pet_id	String	Foreign Key	References PET collection
product_id	String	Foreign Key	References PRODUCT collection
reminder_date	DateTime	-	Reminder notification date
status	String	-	Reminder status

10. ORDER Collection

Description

Stores order information created by users.

Attribute	Data Type	Key Type	Description
order_id	String	Primary Key	Unique order identifier
user_id	String	Foreign Key	References USER collection
subscription_id	String	Foreign Key	References SUBSCRIPTION collection

total_amount	Decimal	-	Total order amount
status	String	-	Order status
payment_status	String	-	Payment status
order_date	DateTime	-	Order creation date

11. ORDER_ITEM Collection

Description

Stores products included in orders.

Attribute	Data Type	Key Type	Description
order_item_id	String	Primary Key	Unique order item identifier
order_id	String	Foreign Key	References ORDER collection
product_id	String	Foreign Key	References PRODUCT collection
quantity	Integer	-	Product quantity
unit_price	Decimal	-	Product unit price

Physical Schema

Database Technology

The system uses MongoDB as a NoSQL database management system.

Physical Design Considerations

- Collections are indexed to improve query performance.
 - Frequently searched fields such as email and category are indexed.
 - Relationships are maintained using referenced document IDs.
 - Data validation rules ensure accurate data storage.
 - Collections are optimized for scalability and performance.
-

Normalization Considerations

The database schema follows normalization principles to reduce redundancy and improve data consistency.

Applied Normalization Techniques

- Independent entities are separated into individual collections.
 - Junction collections are used for many-to-many relationships.
 - Repeated product and order data are separated into item collections.
 - Foreign keys maintain relationships between collections.
-

Benefits of Normalization

- Reduced data redundancy
- Improved data consistency
- Easier maintenance and updates
- Better scalability and organization

11. Literature Review

2.1 Feedback & Evaluation

During the mid-project evaluation, our instructor, Mr. Ebrahim Eltbakh, provided valuable feedback on the current progress of the Zootopia Pet Care E-Commerce Platform. The evaluation highlighted several strong points and identified areas for further development.

Positive Feedback:

- **Database Structure & Personalization:** Strong appreciation for the well-organized MongoDB schemas and collections. Linking pet data (type, age, breed, health status) to user profiles successfully enabled accurate product personalization.
- **UI/UX & Brand Identity:** The clean and minimalist design approach was highly praised. The chosen "Baby Blue & White" color palette effectively communicates the sense of comfort, cleanliness, and trust required for a pet care platform.

Areas for Adjustment:

- **Server Response Time:** A directive to optimize the Node.js/Express endpoints responsible for fetching products, ensuring a seamless and fast browsing experience for users without lag.

- **Pixel-Perfect Execution:** A strong recommendation to coordinate closely with the UI designers to ensure the CSS implementation strictly matches the Figma mockups and layouts.

2.2 Suggested Improvements & Practical Steps

As a team of junior developers aiming to build a robust and professional portfolio, we have implemented and plan to continue practical steps that elevate the platform's performance, maintainability, and organizational quality:

A. Backend: Code Quality & Database Efficiency

- **Modular & Reusable Code:** We successfully migrated the backend to a complete MVC architecture utilizing ES Modules to ensure component independence. We plan to isolate repetitive validation and authentication middlewares, refactoring them into shared centralized services to reduce code duplication.
- **Database Queries Optimization:** We secured and optimized retrieval queries within the product controller (`productController.mjs`). We also plan to apply database indexing on frequently searched fields (e.g., Category, Brand) alongside server-side pagination to accelerate Node.js response times.

B. Frontend: Interface Smoothness & Instant Interaction

- **React SPA & Client-Side Routing:** We fully leveraged React.js and React Router to deliver an authentic Single Page Application (SPA), enabling instant client-side routing across the platform's 13 pages without full browser reloads.
- **Real-Time Cart & Bundle Synchronization:** We developed a global state management solution (`Cart Context`) to instantly synchronize the shopping cart state with the interactive Bundle Builder tool. We plan to implement unified visual transitions and Skeleton Loaders across all data-fetching endpoints.

C. Project Management & GitHub Collaboration

- **Preventing Code Conflicts:** To efficiently coordinate tasks among our six team members, we structured our workflow using dedicated feature branches and established isolated submission directories for each developer.
- **Pull Request Reviews:** We plan to enforce a strict Pull Request (PR) review mechanism via GitHub, ensuring no code is merged into the main branch unless it has been thoroughly reviewed and tested by at least two other team members.

2.3 Final Grading Criteria & Compliance

Based on the official DEPI R4 Project Guidelines and the evaluation standards set for the discussion committee, the table below outlines these criteria and demonstrates how our Zootopia Pet Care platform successfully complies with each requirement:

Grading Criterion	Detailed Description	Project Compliance & Deliverables
Documentation	Completeness of software analysis files, diagrams, and system structural/data flow charts.	- Established a clear documentation structure including detailed MongoDB Schemas. - Fully documented system APIs and router paths within the Express/Node.js server. - Maintained and organized meeting minutes to track continuous development.
Implementation	Quality of source code execution (Frontend & Backend), and the operational efficiency of core features, products, and interactive services.	- Fully migrated the backend to an MVC architecture utilizing ES modules (.mjs) to maximize efficiency. - Built a secure "Forgot/Reset Password" system utilizing bcrypt for token encryption. - Developed an interactive Cart Context in React, dynamically upgrading the shopping experience.

Testing	The testing plan and details followed to ensure the platform is stable and free of bugs.	- Integrated smart error handling and offline API Client fallbacks in React to prevent UI crashes and ensure continuous operation. - Thoroughly tested route efficiency, custom filter queries, and product fetching to guarantee stability.
Presentation	The team's ability during the final discussion to explain solutions and clearly define each member's role and responsibility.	- Designed an interactive presentation interface containing independent files for each of the six team members (e.g., index-mostafa.html, index-rawaa.html) compliant with the Figma frames. - Facilitated a seamless review of each developer's specific contribution and code.

12. System Deployment & Integration

The **Zootopia** project's software architecture relies on a comprehensive, integrated environment based on the **MERN Stack**. This enables the development of a highly efficient and ultra-responsive web application by unifying the programming language (JavaScript/TypeScript) across all application layers.

1. MERN Stack Architecture

The system consists of four foundational pillars that communicate seamlessly:

- **MongoDB:** A NoSQL database that stores data in JSON-like documents. It was utilized for its high flexibility and speed in processing users, products, orders, and subscriptions.

- **Express.js:** A lightweight web application framework running on top of Node.js, used to manage the routing system and build Application Programming Interfaces (APIs) efficiently with low resource consumption.
- **React:** An interactive frontend user interface library used to build a Single Page Application (SPA), achieving blazing-fast navigation between store components without requiring full page reloads.
- **Node.js:** A runtime environment for executing JavaScript code on the server (Backend), acting as the core server engine and controlling logical operations.

2. Client-Server Integration

Communication and data exchange between the frontend and backend strictly follow these standards:

- **RESTful APIs:** The backend functions as an independent service provider, outputting data in **JSON** format. The frontend utilizes query libraries (such as **Axios** or the **Fetch API**) to send HTTP requests (GET, POST, PUT, DELETE) to retrieve or manipulate data.
- **Authentication & Security (JWT - JSON Web Tokens):**
 - Upon successful user identity verification, the server issues an encrypted and digitally signed JWT.
 - The React frontend stores this token and automatically includes it in the header of subsequent requests (**Authorization: Bearer <Token>**) to verify permissions and grant access to protected routes, such as pet management and checkout processes.
- **Cross-Origin Resource Sharing (CORS):** A CORS middleware was configured within Express to open secure, controlled communication channels between the frontend port (e.g., **localhost:5173**) and the server, preventing unauthorized external attacks and allowing only trusted requests.
- **Data Modeling (Mongoose ODM):** Mongoose was employed as an Object Data Mapper to define clear data structures (Schemas) and ensure rigorous input validation and data integrity before saving records to the MongoDB database.

3. Deployment & Environment Configuration

- **Environment Variables (Dotenv Config):** To prevent the leakage of sensitive data, database connection strings (**MONGO_URI**) and digital signature keys (**JWT_SECRET**) were isolated in local **.env** files, completely separated from the pushed repository code.
- **Git Branching Strategy:** Feature pushes to the repository were organized using dedicated Git branches for team members. This structured approach facilitates conflict resolution prior to final merging, ensuring a stable and thoroughly tested codebase.

4. Environment Variables Configuration

To guarantee the system runs correctly in a local environment and to protect sensitive data from appearing in the source code, the **.env** configuration file was utilized in the backend to define the following default variables:

1. **Local Server Port (PORT):**

- **Value:** 5000
 - **Usage:** Defines the software port where the Express server listens for incoming HTTP requests from the frontend.
 - 2. **Database Connection String (MONGO_URI):**
 - **Value:** mongodb://127.0.0.1:27017/petcare
 - **Usage:** Connects the backend to the LocalHost database specifically named petcare to store and process all store and pet-related data locally.
 - 3. **Authentication Secret Key (JWT_SECRET):**
 - **Value:** yoursecretkey
 - **Usage:** The confidential encryption key used to sign and verify JWT tokens for user sessions, ensuring a high standard of security.
 - 4. **Frontend Client URL (CLIENT_URL):**
 - **Value:** http://localhost:5173
 - **Usage:** Specifies the React application URL to configure CORS policies on the server, permitting the frontend to securely communicate with the backend without browser-level restrictions.
 - 5. **Node Environment (NODE_ENV):**
 - **Value:** development
 - **Usage:** Activates local Development Mode, enabling detailed error outputs and console logs to streamline tracking and debugging during the development cycle.
-

4. System Implementation

This phase outlines the practical execution steps and software architecture implemented to build the **Zootopia** platform. The development process strictly followed the Separation of Concerns design pattern to ensure code maintainability, clean organization, and future scalability.

4.1 Project File Architecture

The project was divided into two main repositories to independently couple the frontend and backend environments:

1. Backend (Node.js/Express MVC): The server was designed following the **MVC (Model-View-Controller)** architectural pattern to modularize processing operations as follows:

- **models/:** Contains database table definitions and data type mapping (e.g., `user.mjs`, `pet.mjs`, `product.mjs`).
- **controllers/:** Houses the core business logic to process data, handle requests, and return appropriate client responses.
- **routers/:** The endpoints responsible for receiving HTTP requests and routing them to the corresponding controller.
- **middleware/:** Intermediary functions for security management (e.g., Token verification) and global error handling.

2. Frontend (React): The client-side project was organized into highly reusable components:

- **components/:** Shared UI elements such as the Navbar, Footer, Product Cards, and Pet Profile Cards.
- **pages/:** The primary application views (e.g., Shop, Cart, Account Dashboard, Add Pet, Checkout).
- **context/ & api/:** Directories dedicated to managing global states (logged-in user state, cart state) and handling server communications using the **Axios** library.

4.2 Database Models & Validation

Live data schemas were built using **Mongoose** with strict validation rules to prevent the insertion of invalid or corrupted data:

- **User Model:** Contains fields for name, email (enforced as unique and automatically lowercased), Bcrypt-hashed passwords, and an array of saved shipping addresses.
- **Pet Model:** Stores pet details categorized by specific types and sizes (using Enum validation) and references the owner's ID to establish a relational link.
- **Product Model:** Encompasses all product information, including allowed categories, original and discount prices, unique SKU generation, nutritional weights, and customer reviews.
- **Cart & Order Models:** Manages the user's live calculating cart, checkout flow paths, shipping addresses, and dynamic delivery statuses.

4.3 Core Business Logic

Several key features were programmed into the system to enhance interactivity and buyer engagement:

1. Dynamic Bundle Discounts:

A programmatic function utilizing a Mongoose **pre-save** hook automatically calculates the item quantity before saving the cart. Based on this, dynamic discounts are applied:

- 2 Items = **5%** Discount
- 3 Items = **10%** Discount
- 4 Items = **15%** Discount
- 5+ Items = **20%** Discount

2. Smart Subscriptions:

The system allows users to convert any cart item into a recurring subscription by specifying the delivery frequency (e.g., every 2 or 4 weeks). Upon checkout, the server generates a separate **Active Subscription** document to calculate the next delivery date and automatically update its status.

4.4 Security & Route Protection

The software system is fully secured against common vulnerabilities using the following methods:

- **Password Hashing:** User passwords are encrypted upon registration or update using the **Bcrypt** algorithm with 10 salt rounds, ensuring plain-text passwords are never stored in the database.
 - **Private Routes:** An **authMiddleware** was implemented to intercept incoming request headers. It verifies the presence of a valid JWT token for protected routes (such as the Shopping Cart or User Dashboard). If the token is missing or invalid, the request is immediately rejected with a **401 Unauthorized** status.
-

5. Testing & Quality Assurance

To ensure the platform is free of software bugs and provides a secure, stable user experience, a systematic testing strategy was applied, covering various system aspects from the database layer to the final frontend interfaces.

5.1 Testing Levels Conducted

Our testing approach was divided into three main levels to ensure comprehensive coverage:

1. Unit Testing:

- Core system functions were tested in isolation to verify correct behavior.
- **Example:** Verifying that the password encryption algorithm (**bcrypt**) operates correctly before saving user data, and ensuring hashed tokens match successfully via the **comparePassword** function.

2. Integration & API Testing:

- Tools like **Postman** and **Thunder Client** were utilized to test API endpoints both individually and collectively.
- Verified request handling and the return of appropriate JSON responses (e.g., testing registration, login, and fetching categorized product lists).
- Validated session binding and JWT token integrity when requesting protected resources that require active authentication.

3. System & End-to-End (E2E) Testing:

- A complete end-user journey was simulated on the browser to ensure seamless component integration.
- **Simulated User Flow:** Account Creation → Login → Product Browsing → Adding to Cart → Increasing item quantity to trigger the automatic Bundle discount → Selecting recurring delivery (Subscription) → Entering shipping address and completing Checkout → Reviewing the Orders & Subscriptions dashboard to confirm successful logging.

5.2 QA & Input Validation

Programmatic controls were embedded to prevent operational errors or logical vulnerabilities originating from user inputs:

- **Form Validation:** Prevented frontend form submissions without fulfilling mandatory fields, alongside password strength checks (minimum of 8 characters) and strict email format validation.
- **Quantity & Stock Verification:** Implemented constraints preventing the addition of negative quantities or requesting amounts exceeding the actual inventory. An immediate notification is displayed, and the purchase button is disabled (Out of Stock) if items are unavailable.
- **Responsive Testing:** Interfaces were rigorously tested in Chrome using Developer Tools (DevTools) on simulated smartphone (iOS & Android) and tablet screens to guarantee visual structural stability and prevent overlapping elements or buttons.

5.3 Graceful Error Handling

The system was designed to handle unexpected exceptions gracefully, containing errors and guiding the user without crashing:

- In the event of a server disconnection, the client interface displays user-friendly alert notifications prompting a retry, rather than abruptly crashing the application or rendering a blank screen.
 - The backend captures errors and returns the appropriate HTTP status codes (e.g., 400 for Bad Requests, 404 for Not Found, and 500 for Internal Server Errors).
 - Error details are logged into a local `err.log` file to facilitate developer review and debugging without exposing sensitive code details or stack traces to the end-user.
-

Appendix: Diagram Placeholders

UML & Flow Diagrams

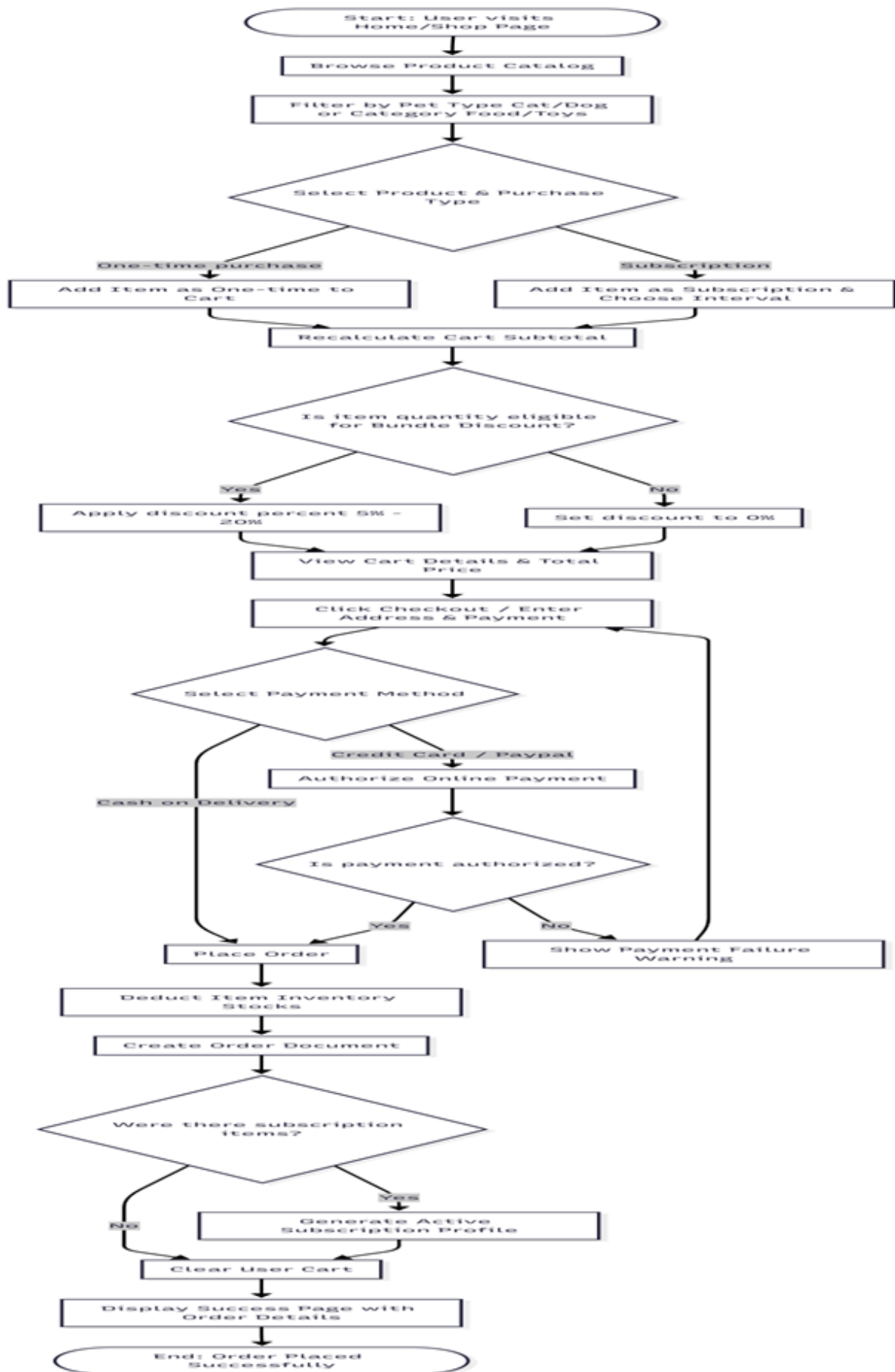
- Activity Diagram

Description

This Activity Diagram illustrates the workflow of the shopping and checkout process. It shows how users browse products, manage their shopping cart, proceed to checkout, select payment and delivery options, and complete an order successfully.

Main Functionalities

- Browse products.
- Add or remove items from the cart.
- Review cart contents.
- Complete checkout.
- Confirm order placement.



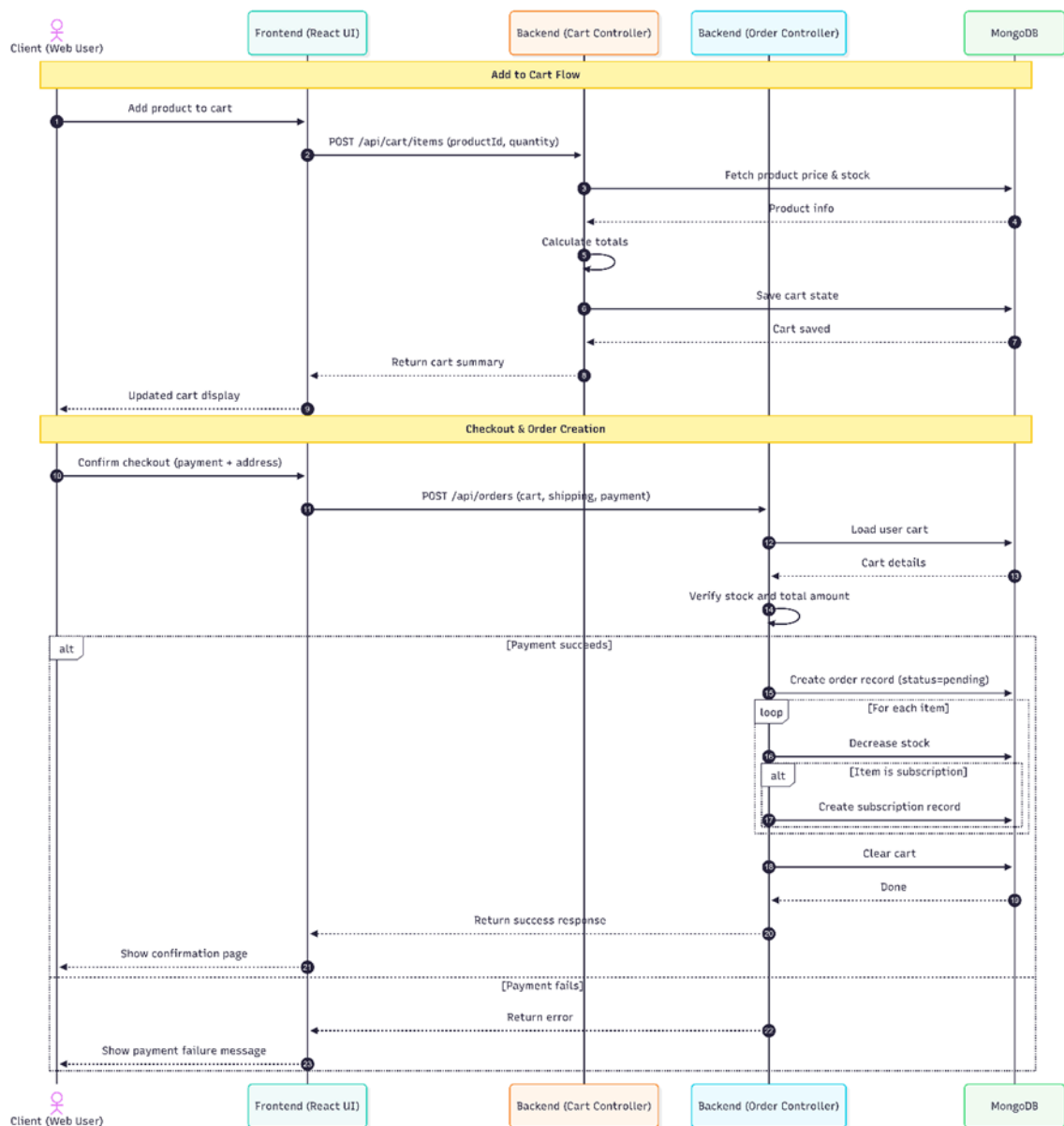
• Cart Checkout Sequence

Description

This Sequence Diagram illustrates the interactions involved when a user completes the checkout process, creates an order, and optionally subscribes to recurring product deliveries.

Main Functionalities

- Checkout process.
- Order creation.
- Payment processing.
- Subscription creation.
- Order confirmation.



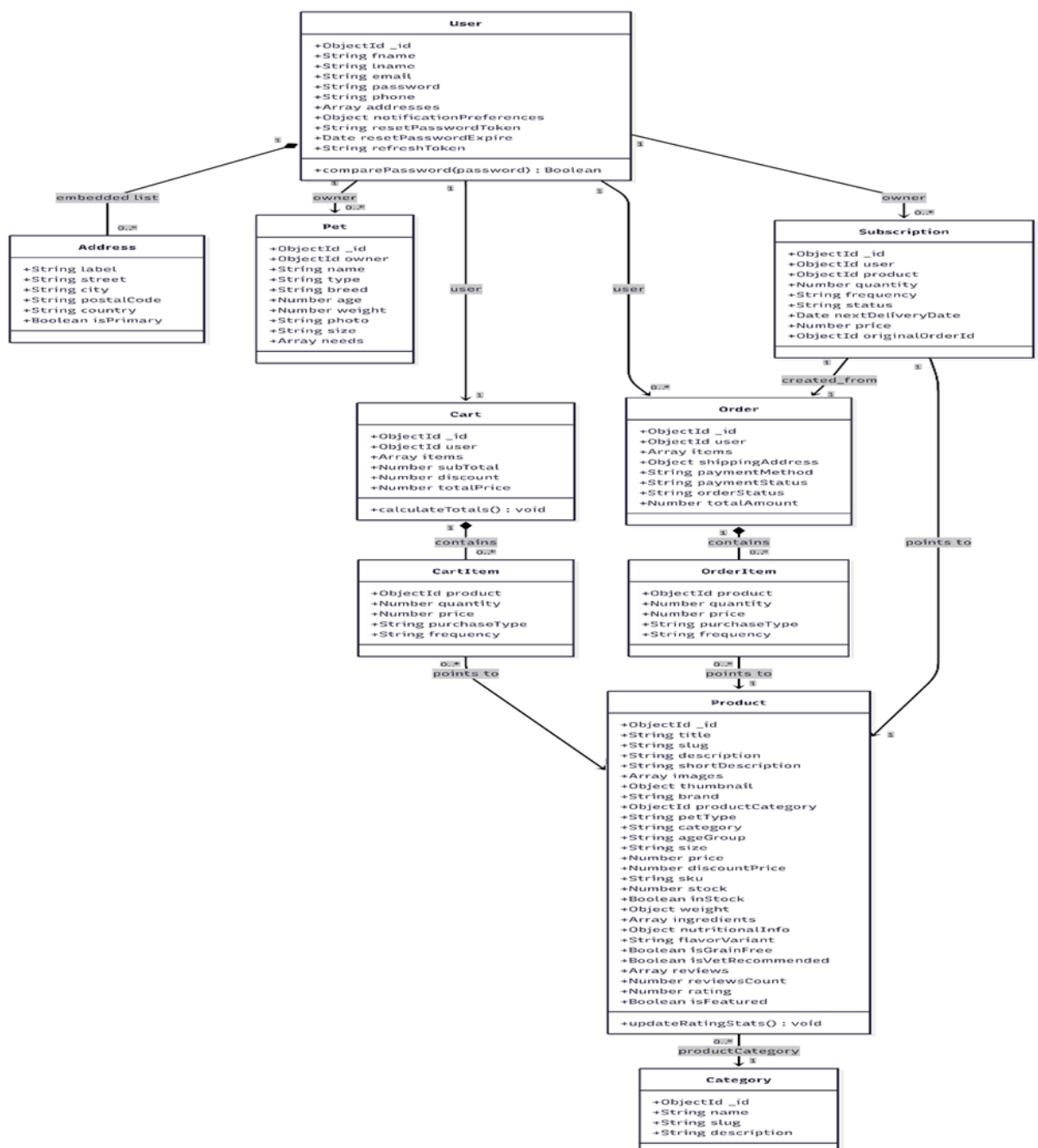
• Class Diagram

Description

The Class Diagram illustrates the static structure of the Pet Care E-Commerce Platform. It represents the main classes, their attributes, methods, and the relationships between them. The diagram provides an overview of how different system components interact to support user management, pet management, shopping, orders, subscriptions, and notifications.

Main Functionalities

- Represents the core system classes.
- Shows relationships between entities.
- Defines attributes and operations of each class.
- Supports maintainability and system scalability.



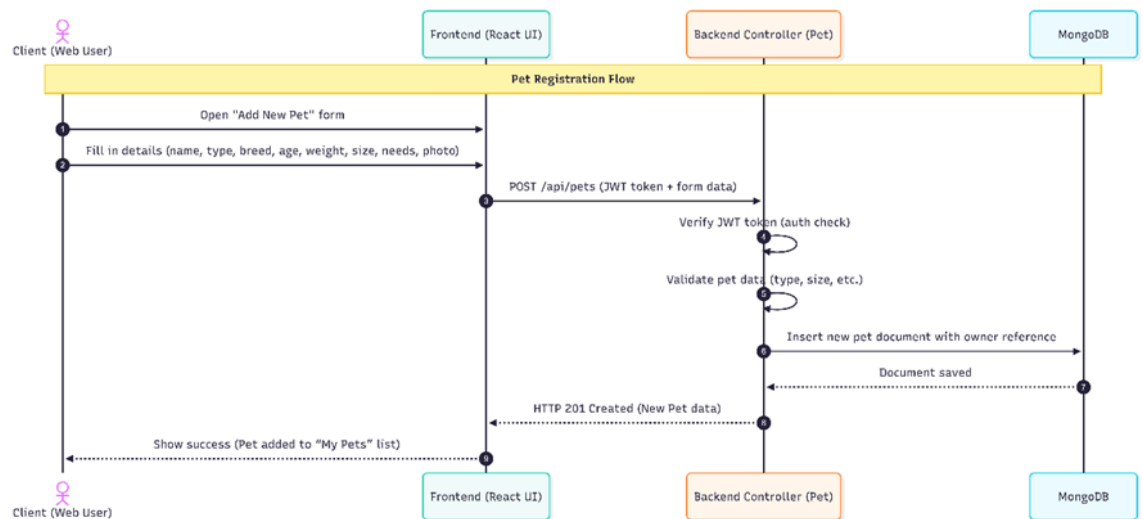
• New Pet Registration Sequence

Description

This diagram shows the sequence of interactions required to register a new pet. The user submits pet information, the backend validates the data, stores it in the database, and confirms successful registration.

Main Functionalities

- Enter pet information.
- Validate pet data.
- Save pet profile.
- Confirm successful registration.



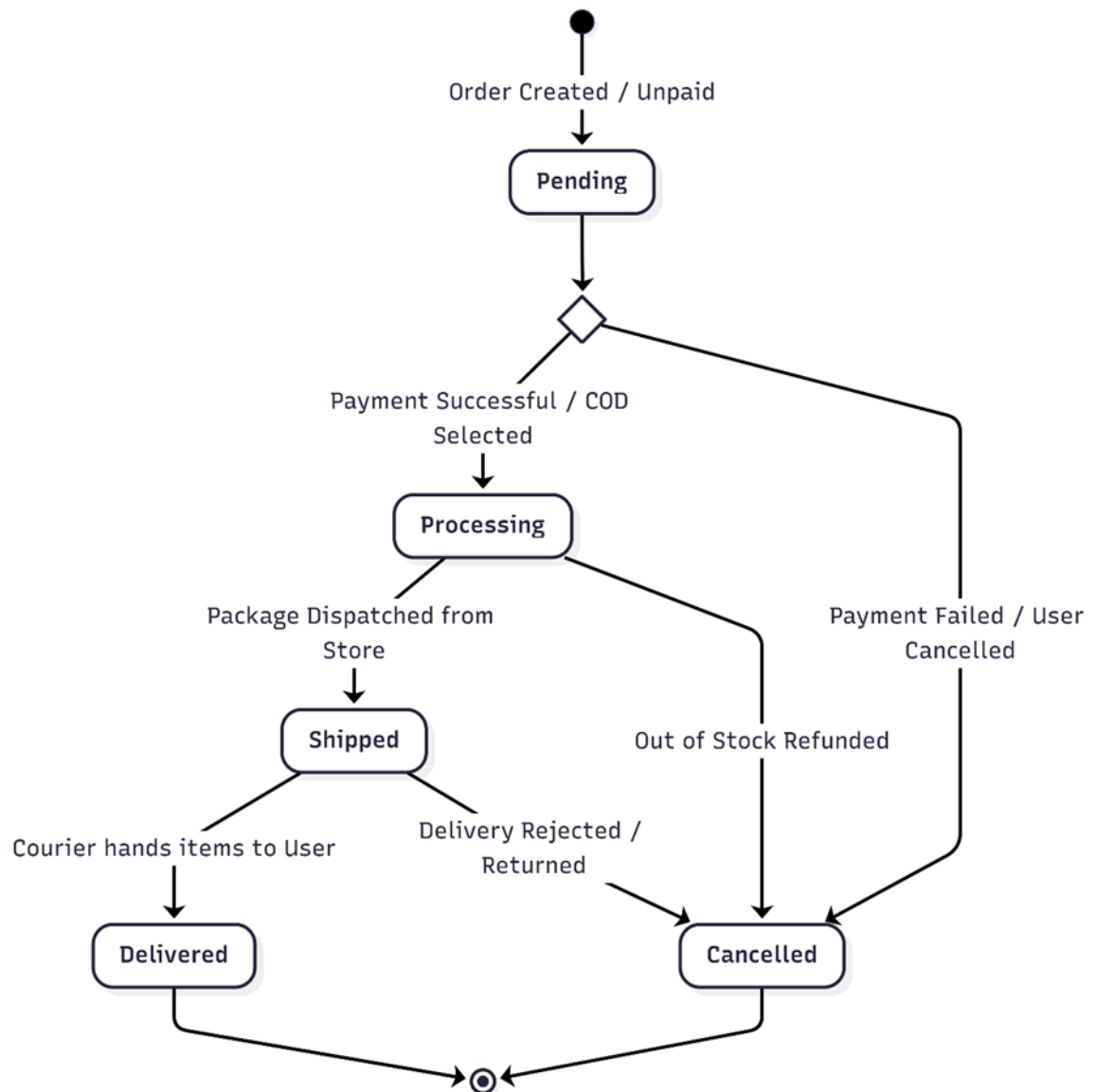
• Order Lifecycle State

Description

The Order Lifecycle State Diagram represents the different states an order passes through, starting from creation until successful delivery or cancellation.

Main Functionalities

- Pending order.
- Processing.
- Shipped.
- Delivered.
- Cancelled.



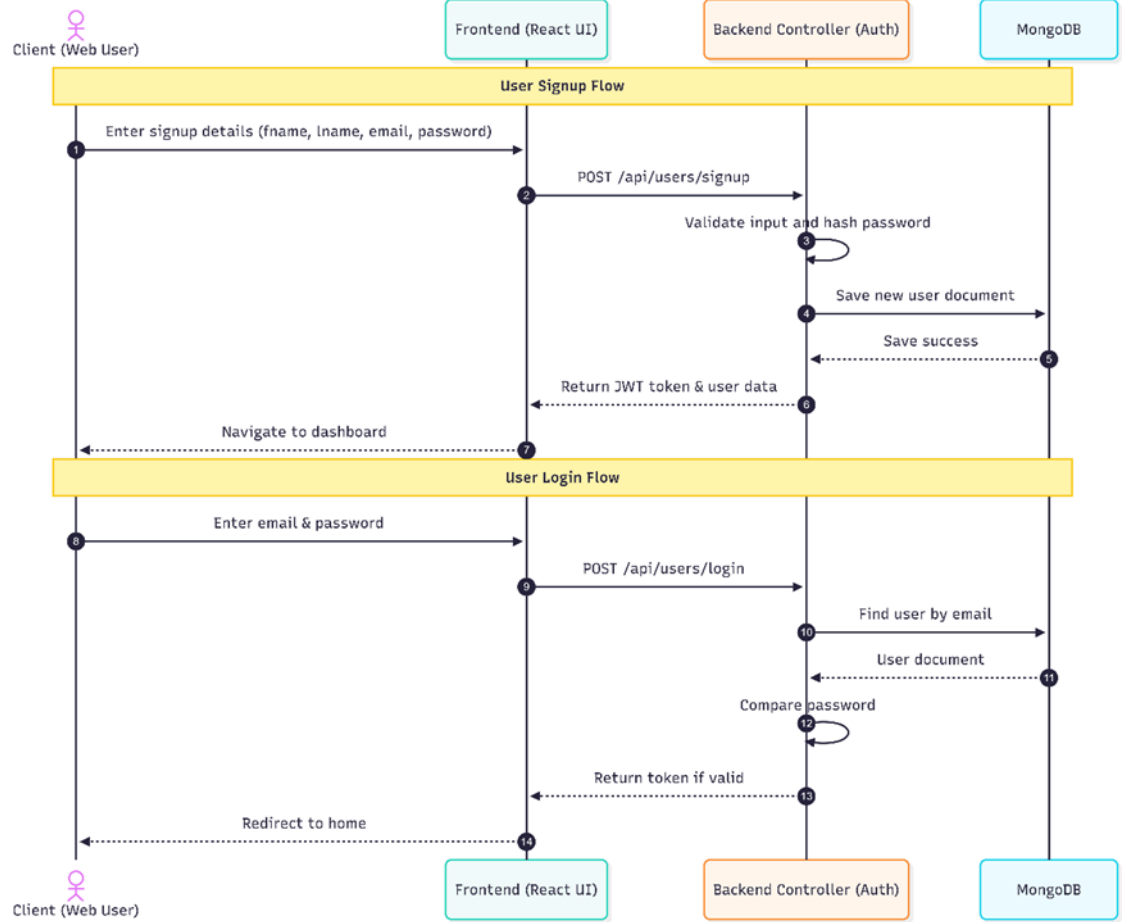
• Signup/Login Sequence

Description

This Sequence Diagram illustrates the interaction between the user, frontend, backend, and database during user registration and login. It demonstrates the authentication process, credential validation, and secure access using JWT tokens.

Main Functionalities

- User registration.
- User authentication.
- Password validation.
- JWT token generation.
- Secure login session.



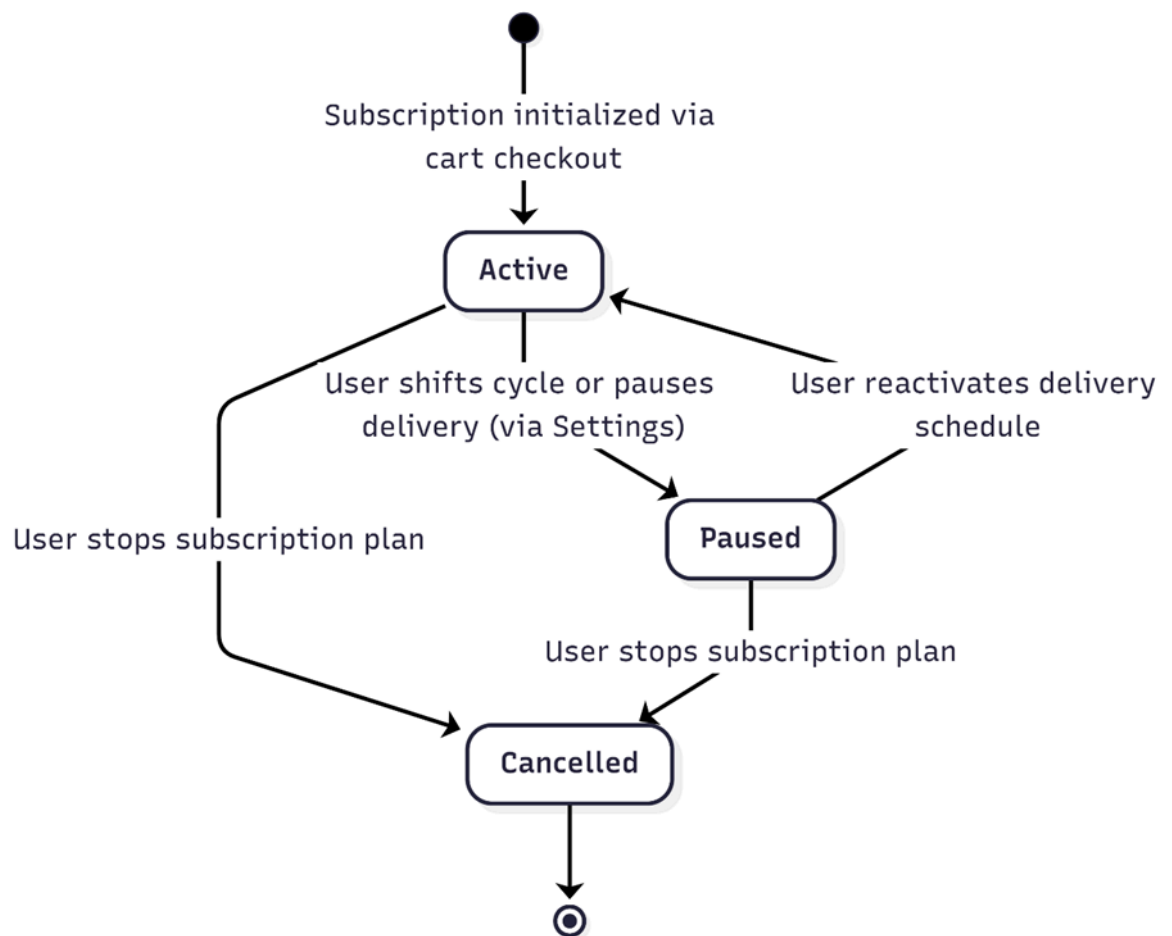
• Subscription State

Description

This State Diagram illustrates the lifecycle of a product subscription. It shows how subscriptions transition between different states based on user actions and scheduled events.

Main Functionalities

- Create subscription.
- Activate subscription.
- Pause subscription.
- Resume subscription.
- Cancel subscription.



DFD

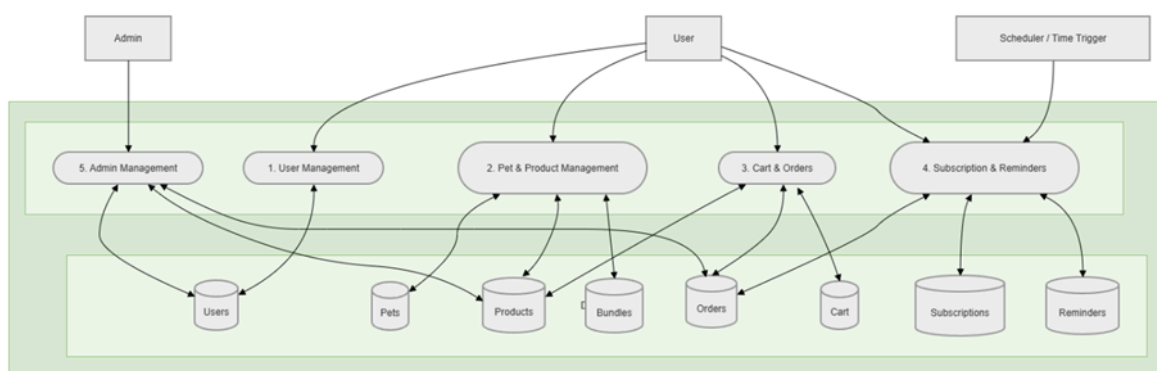
• Context

Description

The Context Diagram provides a high-level overview of the entire system. It illustrates the interaction between external entities and the Pet Care E-Commerce Platform through major data flows.

Main Functionalities

- Show system boundaries.
- Identify external actors.
- Represent major data flows.



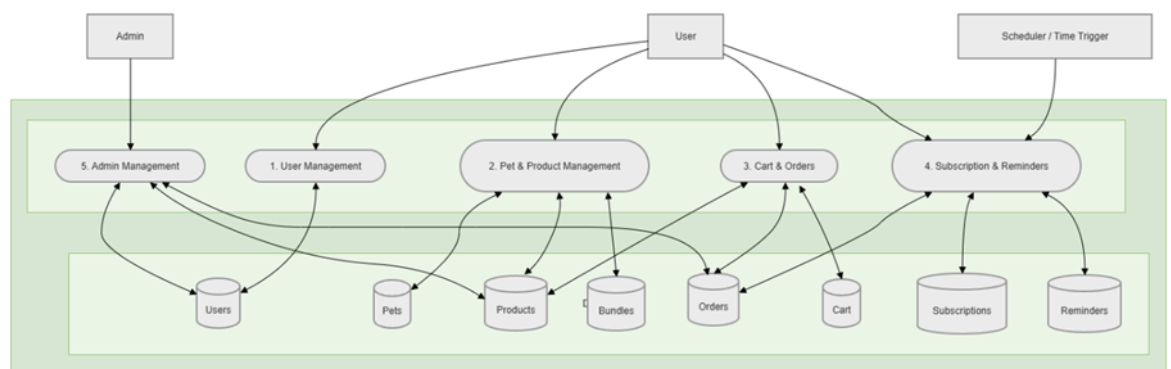
• Level-0

Description

The Level-0 Data Flow Diagram decomposes the system into its major processes and demonstrates how data moves between users, processes, and data stores.

Main Functionalities

- User management.
- Product management.
- Order management.
- Subscription management.
- Notification management.



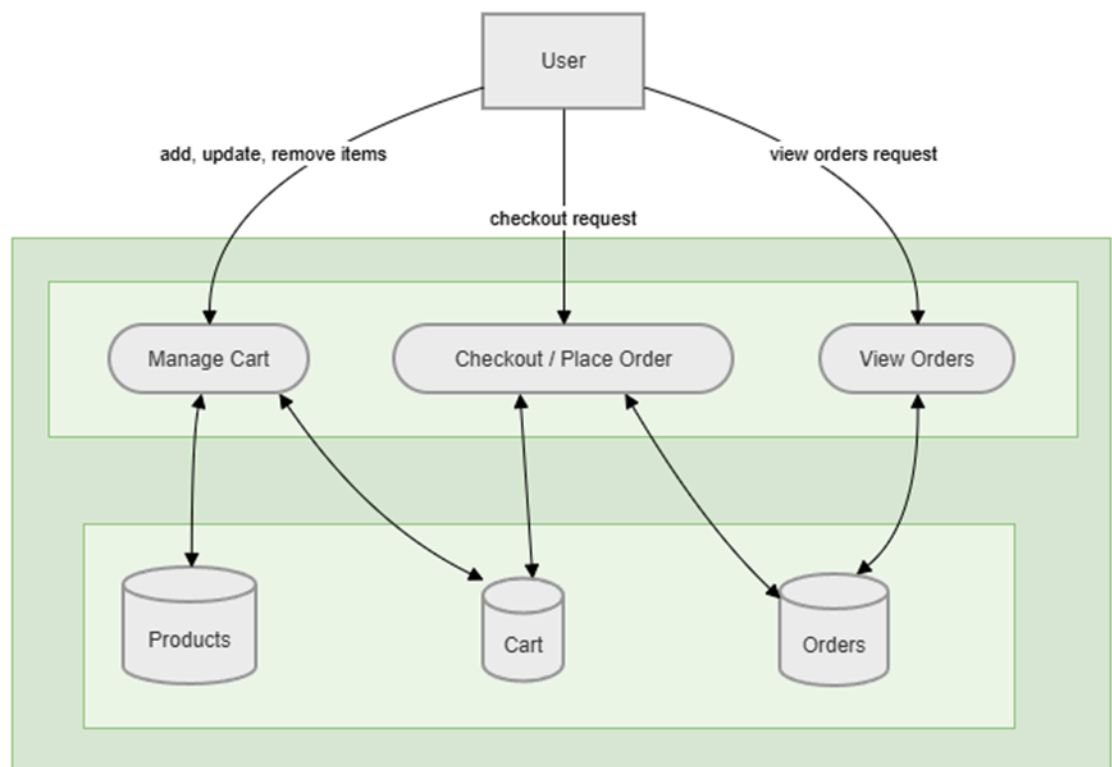
• Level-1 Cart

Description

This diagram illustrates the internal processes involved in shopping cart management and order processing, from adding products to completing purchases.

Main Functionalities

- Add products to cart.
- Update cart.
- Create orders.
- Store order details.



Level 1 - Cart and Order Processing

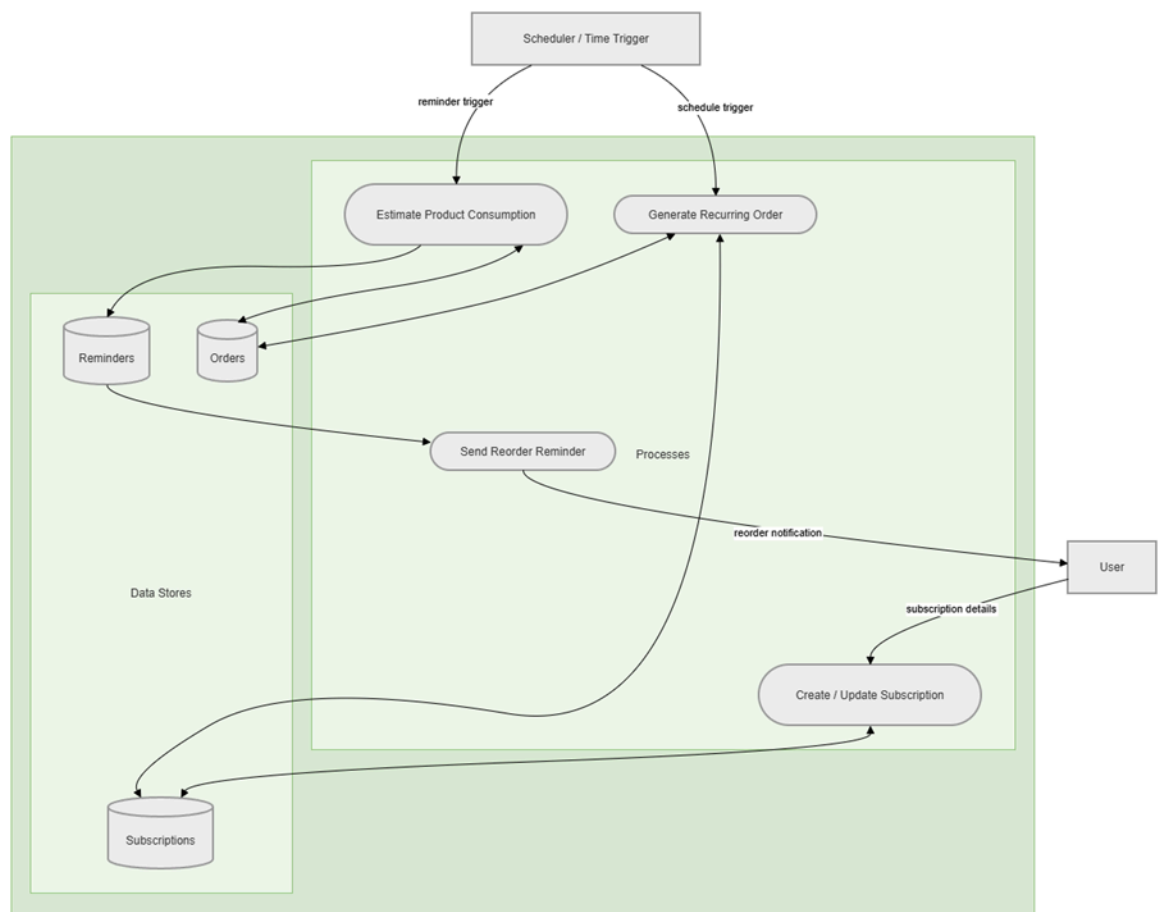
• Level-1 Subscription

Description

This diagram illustrates how recurring subscriptions and automated reminder notifications are processed within the system.

Main Functionalities

- Create subscriptions.
- Manage delivery schedules.
- Generate reminders.
- Send notifications.



Level 1: Subscription + Reminder

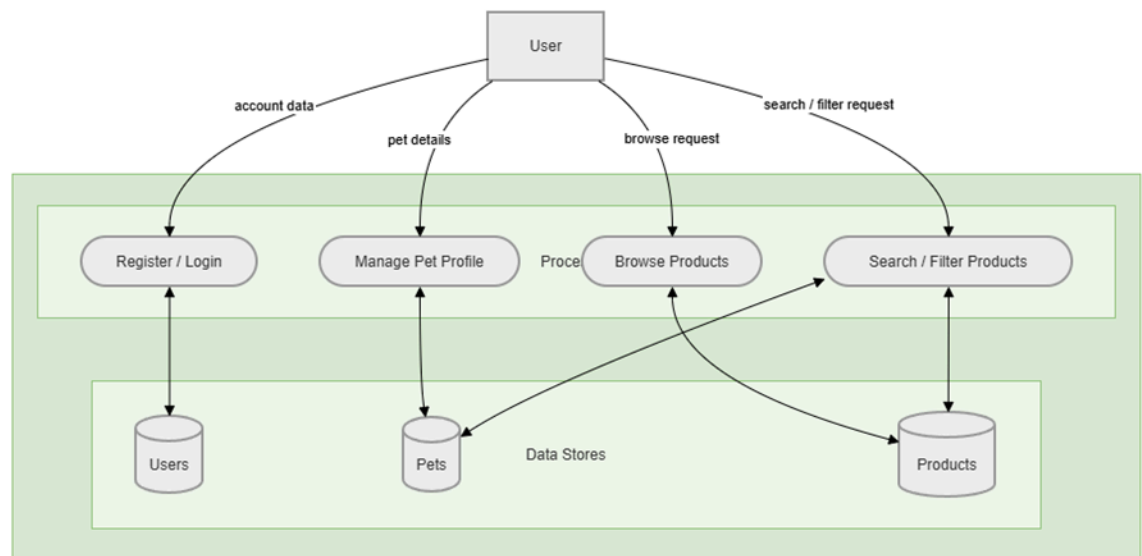
• Level-1 User

Description

This Level-1 DFD explains how the system manages user information, pet profiles, and personalized product filtering based on pet characteristics.

Main Functionalities

- Manage user profiles.
- Manage pet profiles.
- Filter products.
- Retrieve personalized recommendations.



Level 1 - User, Pet Profile, and Product Filtering